1 Waspmote

Waspmote is a proprietary mote for Wireless Sensor Networks. Waspmote is based on a modular architecture, therefore the developer can integrate only the modules needed in each device. In the experiment described in this section, we have used waspmote PRO v1.2 with ATmega1281 Microcontroller and 8KB SRAM. Additionally, to achieve consumption communications evaluations Wifi and Bluetooth module have been plugged to socket0 on the board.

1.1 Measurement procedure

The measurement process has been addressed using an Arduino Shield connected to an Arduino. Originally, the Arduino Shield has been deployed as teaching material for real time systems, robots programming and automatic control subjects, although its design makes suitable to measurement low energy consumption executions, as cryptographic primitives and communications of information with a limited size. Therefore, due to its specific characteristic, we considered as convenient to measurement energy consumption of code executed in the waspmote. The shield has a 64kB SRAM memory to store data acquired, therefore each execution of an experiment are limited to approximately 1.8 seconds of data records. This limitation has to been taken into consideration to design the sketch of the experiment, i.e. connection and disconnection of bluetooth communication are too time consumption to include in our experiments. A detailed description of measurement Arduino Shiled can be found in [1].

Waspmote has eight digital outputs in the sensor connector pins. The figure 1 shows the Waspmote connected to measurement Arduino Shield. We have established two connection between Waspmote and the Arduino Shield, the USB connector to feed the WaspMote, and the green wired to exchange the signal control. The signal control is used to advise when the Arduino Shield must to start and stop the measurement process. If the developer wants to measure the consumption of a code executed in the mote, the following C code must to be executed in the waspmote.

Listing 1: Example of primitive execution

```
digitalWrite(DIGITAL1,LOW);
  <Code to measure>
digitalWrite(DIGITAL1,HIGH);
```

Previously, in the setup section of the Waspmote code the digital output one has been chosen as output signal control, according to the green wire connection shown in the figure 1.

Listing 2: Configuration of digital one pin as output

```
  pinMode(DIGITAL1,OUTPUT);
```

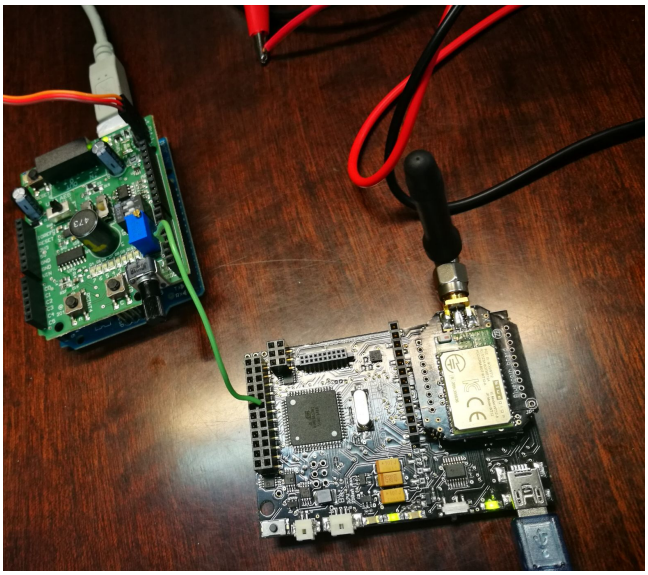

Figure 1: Bluetooth measurement

1.2 Cryptography

Waspmote offers only two cryptographic algorithms, symmetric and asymmetric, AES and RSA respectively.

In the case of AES three key length are supported 128, 192 and 256. Regarding to mode and padding, on one hand, only ECB and CBC are implemented and on the other hand PKCS5 and ZERO are included. We develop a waspmote sketch to measure the same configuration (mode, padding) with three different key length: 128, 192 and 256. The figure 2 shows an example of AES ECB PKCS5 configuration. The control signal, the blue line, has two states, zero and one. The control signal is set to one, *HIGH*, when the cryptographic primitive is executed. The time consumed is calculated using the start and end point of control signal. Furthermore, the energy consumed for the code is the derivatives of consumption function between the initial and final point of control sign. Among the primitives execution, we set a delay which matches with the control sign to zero. The greatest delay is set at the end of the execution of 256 key length, to identify each execution.

Evaluations of the symmetric primitive cryptography AES, is configured with three different keys, two modes and two padding configuration. Each configuration (key length, mode and padding) is evaluated encrypting three different length string: 10, 100 and 200 bytes. Table 1 details the execution time of each configuration of AES encryption. In this case, the measurement is performed using software procedure, that is to say, calling the function *millis*. The *millis* function measures the execution time since Waspmote is turned on, therefore if we set an instruction between two *millis* invocation we can calculate

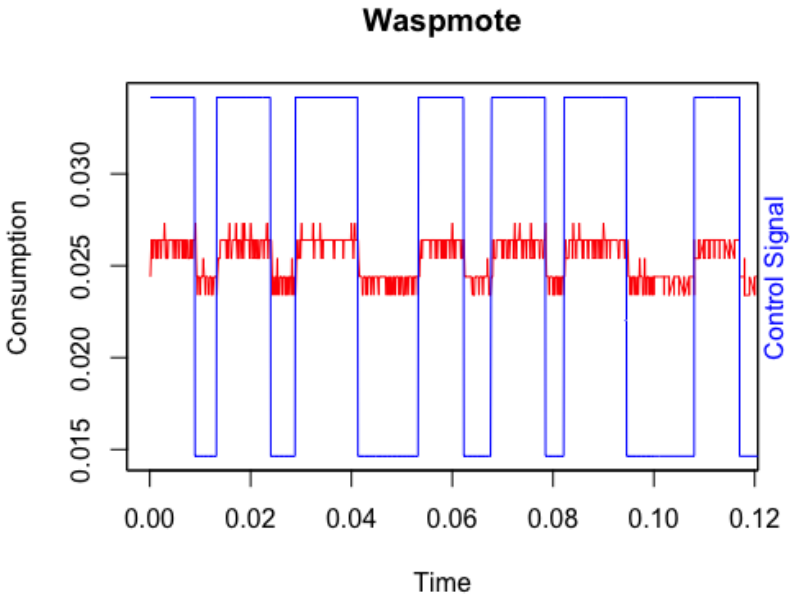

Figure 2: AES measurement data

how milliseconds spends the execution of the code to evaluate.

Time (milliseconds)			**Data length**		
Key Length	**Mode**	**Padding**	**10**	**100**	**200**
128	ECB	PKCS5	1,00	8,60	15,80
		ZEROS	1,00	8,40	15,60
	CBC	PKCS5	1,00	9,00	16,70
		ZEROS	1,00	8,60	16,70
192	ECB	PKCS5	1,00	10,00	18,70
		ZEROS	1,00	10,00	18,60
	CBC	PKCS5	1,00	10,70	20,00
		ZEROS	1,00	10,30	20,00
256	ECB	PKCS5	2,00	12,00	22,00
		ZEROS	2,00	12,00	22,00
	CBC	PKCS5	2,20	12,00	23,20
		ZEROS	2,00	12,20	23,10

Table 1: AES time software

On the other hand, table 2 details the execution time calculated using the

Arduino Shield. Specifically the time is the interval between the activation and deactivation of the control signal. At first sight, there is not significant difference between hardware and software measurement,even so we have performed a statistical test that confirm our initial hypothesis.

TIME (milliseconds)			**Data length**		
Key Length	**Mode**	**Padding**	**10**	**100**	**200**
128	ECB	PKCS5	1,33	8,98	16,01
		ZEROS	1,30	8,64	16,74
	CBC	PKCS5	1,42	9,00	16,59
		ZEROS	1,44	9,08	16,96
192	ECB	PKCS5	1,65	10,68	19,94
		ZEROS	1,74	10,56	19,32
	CBC	PKCS5	1,82	10,86	19,97
		ZEROS	1,70	10,82	20,00
256	ECB	PKCS5	2,00	12,31	22,05
		ZEROS	2,03	12,31	22,73
	CBC	PKCS5	2,01	12,37	22,82
		ZEROS	2,19	12,47	23,00

Table 2: AES time hardware

The table 3 shows the energy consumption of each configuration. Our initial evaluation is that there is not significant difference between the consumption of AES encryption using different modes or padding, however the energy increase if the key or data encrypted length is increased. On one hand, an increment of 10 to 100 bytes to encrypt implies 6 times increment in energy consumption, whereas the difference from 100 to 200 bytes is translated to around 1.8 times. On the other hand, the increment of key length is approximately 1.2 times in both case; from 128 to 192 and from 192 to 256 key length. In the case the developer wants to move from less to the most secure key the consumption difference is around 1.5 times.

In our opinion, developer can assume the energy consumption of the most secure key configuration. Furthermore, it is recommendable encrypt using longer data package, i.e. using 256 key length, ECB mode and PKCS5 padding, if the developer needs to encrypt 400 bytes, the consumption will be 44 mJ, otherwise in the case she encrypt four packages of 100 bytes, 48 mJ will be consumed and finally 80 mJ will be consumed if the developer decide to use 10 bytes packages. In other words, it is preferred than developer store data to build higher buffer and the encrypted it, than encrypt short buffers of information.

Furthermore, RSA has supported to encrypt-decrypt and sign-verify, although no key generation is supported due to high computation requirements. The operation works correctly using 512 length key but its performance is erratic with higher length keys. The table 4 shows the time execution in software and hardware, encrypt two different data length. The conclusion of the experi-

Consumption (milliJoules)			Data length		
Key Length	**Mode**	**Padding**	**10**	**100**	**200**
128	ECB	PKCS5	0,033	0,23	0,42
		ZEROS	0,034	0,22	0,42
	CBC	PKCS5	0,036	0,24	0,42
		ZEROS	0,037	0,24	0,44
192	ECB	PKCS5	0,041	0,27	0,52
		ZEROS	0,045	0,27	0,49
	CBC	PKCS5	0,046	0,29	0,51
		ZEROS	0,043	0,29	0,52
256	ECB	PKCS5	0,050	0,32	0,58
		ZEROS	0,053	0,31	0,58
	CBC	PKCS5	0,050	0,33	0,58
		ZEROS	0,056	0,33	0,60

Table 3: AES consumption hardware

ment is that the time and energy consumption do not depend on the data length encrypted.

	Software	**Hardware**	
Data length	**Time**	**Time**	**Consumption**
10	454	455,17	11,48
100	455	454,03	11,26

Table 4: RSA encryption 512 key length

1.3 Communications

Regarding the communications two modules are evaluated, Bluetooth and Wifi. In both cases a string with different length has been sent between the waspmote and an Android smartphone in the Bluetooh case and between the waspmote and a laptop in the Wifi communication. The waspmote is configured as client in both communications therefore, a proof on concept Bluetooth terminal Android application and a Java echo Server have been deployed for Bluetooth and Wifi communications respectively.

The table 5 details the software and hardware time execution and energy consumption in both communication. As in the previous cases, there is not considerable differences between software and hardware time measurement.

Figure 3 shows a comparative between the energy consumption between Bluetooth and Wifi module. In both cases, the establishment phase of the communication are not included in the consumption process, we focus in the communication phase. The chart (figure 3) shows the communications using

	Bluetooth			Wifi		
	Software	Hardware		Software	Hardware	
Data length	**Time**	**Time**	**Cons.**	**Time**	**Time**	**Cons.**
10	100.06	99,42	3,38	3,56	3,10	0,19
100	107,98	107,26	3,62	11,56	10,71	0,65
1000	187,00	186,41	9,65	89,00	89,74	5,75
2000	273,29	273,22	13,28	176,40	176,27	11,43

Table 5: Communication consumption

Bluetooth module consume more than Wifi module, specially between 10 and 1000 bytes, from 3 to 2 times approximately. In the case of 2000 bytes the difference is not as significant as before.

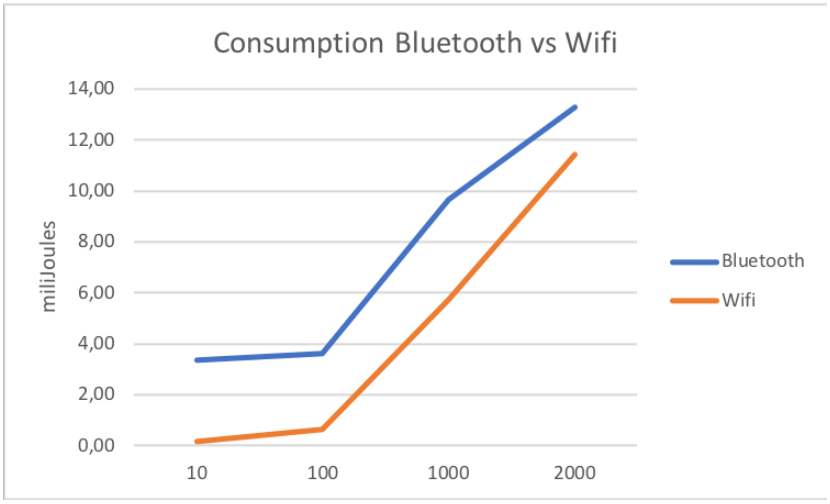

Figure 3: Consumption Bluetooth vs Wifi modules

On the other hand, the time consumed in Bluetooth communications is clearly higher than Wifi communications. Although, the gap between time consumed is reduced when the transmission data length is increased.

If a developer needs to send 2000 bytes in a sole wifi connection the energy consumed is 11.43 mJ as shown in table 5. In case other buffer length is selected the consumption will be 38 mJ for 10 bytes, 13 mJ for 100 bytes and 11.5 mJ for 1000 bytes. It seems that if the developer wants to send large amounts of data is recommendable to use a long buffer in an unique communication than short packages in several transmission, although the consumption gap between 1000 and 2000 bytes is almost despicable.

The same experiment applied to Bluetooth communication shows than a communication of 2000 bytes consumed 13.28 mJ. If other buffer length com-

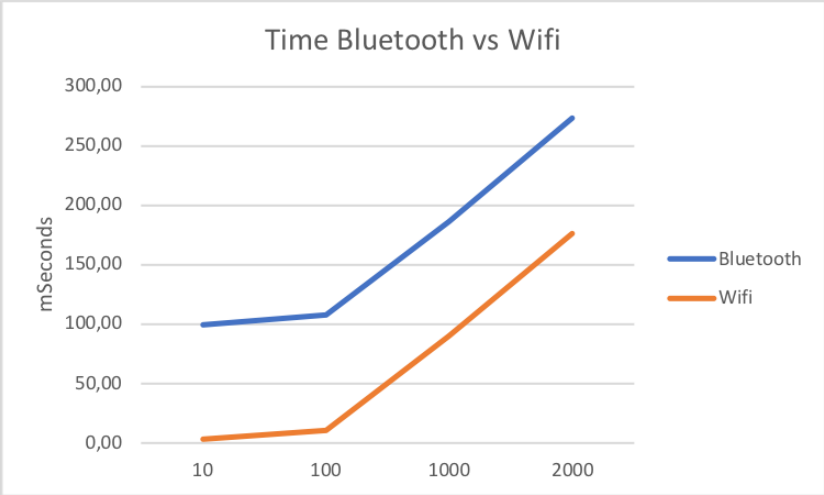

Figure 4: Time Bluetooth vs Wifi modules

munication is used the increment of consumption energy is extremely higher 676 mJ for 10 bytes buffer, 67.6 mJ for 100 bytes buffer and 19.3 mJ for 1000 bytes length. The energy consumed decreased when the buffer length is incremented, wherefore it is recommendable to use the greatest buffer communication in a Bluetooth connection when a large amounts of data must to be sent.

2 Biblio

[1] Gongora, A; Fernández-Madrigal, J.A., Fernández-Madrigal, J.A.; Cruz-Martín, A.; Fernandez de Canete, J. "Shield Arduino de bajo coste para la enseñanza de asignaturas de Ingeniería". II Jornadas de Computación Empotrada y Reconfigurable (JCER2017). Málaga (Spain)